Interrogation Avancée en SQL : Jointures Relationnelles et Regroupements (GROUP BY & HAVING)

Thème principal : Jointures relationnelles (INNER JOIN, LEFT JOIN) et fonctions d'agrégation avec clauses GROUP BY et HAVING en SQL

Synthèse exécutive : Ce cours aborde les mécanismes fondamentaux du langage SQL pour croiser des données issues de multiples tables relationnelles via les jointures (INNER JOIN, LEFT JOIN) et structurer les résultats par groupes au moyen des clauses GROUP BY et HAVING. L'objectif est d'assimiler la logique d'exécution des requêtes complexes, de maîtriser le calcul d'agrégats statistiques (AVG, COUNT, SUM) et d'éviter les pièges classiques tels que la génération involontaire de produits cartésiens ou l'utilisation erronée de fonctions d'agrégation dans la clause WHERE.

1. Les Jointures Relationnelles : Interrogation Multi-Tables (INNER JOIN vs LEFT JOIN)

Dans un SGBDR (Système de Gestion de Base de Données Relationnel), les données sont décomposées en plusieurs tables normalisées. Pour réassembler ces informations lors d'une interrogation, on utilise les jointures dans la clause FROM. L'évaluation de la jointure intervient au tout début du cycle d'exécution de la requête. Si le critère d'équivalence (ex: E.id_etud = N.id_etud) n'est pas explicitement déclaré dans la clause ON, le SGBD effectue un produit cartésien combinant chaque ligne de la première table avec l'ensemble des lignes de la seconde, provoquant une explosion combinatoire et un risque élevé de crash du serveur.

- Normalisation & Équijointure : Séparation des entités pour éviter la redondance et création de liaisons explicites via Clé Primaire (PK) et Clé Étrangère (FK).
- INNER JOIN (Jointure Interne) : Restitue exclusivement les combinaisons de n-uplets satisfaisant le prédicat d'association défini dans la clause ON.
- LEFT JOIN (Jointure Externe Gauche) : Conserve la totalité des enregistrements de la table de gauche et complète par NULL si aucun enregistrement correspondant n'existe dans la table de droite.
- Produit Cartésien (CROSS JOIN) : Résultat d'une jointure sans critère de liaison (clause ON omise), générant une combinaison de toutes les lignes de la table A avec toutes celles de la table B (N x M lignes).

Cas concret enseignant : *Le professeur compare l'INNER JOIN à l'entrée d'un bâtiment sécurisé à l'IUT : seuls les étudiants possédant un badge valide et une inscription enregistrée peuvent entrer. Si une personne manque à l'un des deux côtés, elle est exclue du résultat. À l'inverse, le LEFT JOIN ressemble à la liste officielle d'appel du département Informatique : tous les étudiants inscrits sur le registre apparaissent, et pour ceux qui n'ont pas encore rendu de note, le bulletin affiche la valeur 'NULL'.*

2. Regroupement de Données et Agrégation : Clauses GROUP BY et HAVING

L'analyse statistique impose souvent de partitionner les enregistrements d'une table selon un critère métier (ex: regrouper les notes par matière ou par étudiant). La clause GROUP BY effectue ce partitionnement. Une erreur conceptuelle majeure consiste à essayer d'utiliser la clause WHERE pour filtrer les résultats agrégés (comme WHERE AVG(note) > 10). Cela est physiquement impossible pour le moteur SQL car la clause WHERE filtre les lignes individuelles avant que les groupes ne soient formés et calculés. Pour appliquer une condition restrictive sur une valeur agrégée, SQL impose l'utilisation de la clause HAVING qui s'exécute après le regroupement.

- Fonctions d'Agrégation : Calculs synthétiques réduisant un ensemble de n-uplets à une valeur unique (COUNT, SUM, AVG, MIN, MAX).
- GROUP BY : Subdivise les n-uplets filtrés en sous-groupes homogènes partageant les mêmes valeurs pour les attributs spécifiés.

• Règle d'or du SELECT sous GROUP BY : Toute colonne figurant dans le SELECT doit obligatoirement soit appartenir au GROUP BY, soit être enserrée dans une fonction d'agrégation.

• Clause HAVING : Prédicat de filtrage s'appliquant exclusivement sur les groupes formés et sur les résultats des fonctions agrégées.

• Ordre logique d'exécution : FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY.

Cas concret enseignant : *L'enseignant illustre le fonctionnement en comparant le GROUP BY au tri physique des copies d'examen en amphithéâtre : le professeur rassemble d'abord l'intégralité des copies dans des bannettes distinctes par groupe de TD (ce qui représente la clause GROUP BY). Ensuite, pour chaque paquet, il calcule la moyenne du groupe (fonction d'agrégation AVG). Si l'administration décide de ne retenir que les paquets de TD dont la moyenne dépasse 12/20, c'est la clause HAVING qui joue le rôle de filtre sur les paquets d'examen.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi est-il strictement impossible d'écrire 'WHERE AVG(note) > 10' dans une requête SQL ?
Réponse du professeur : C'est une question de chronologie dans l'ordre d'exécution logique du moteur SQL. La clause WHERE intervient AVANT le partitionnement des données par GROUP BY. Au moment où le WHERE est évalué, les paquets de données n'existent pas encore et la moyenne AVG(note) n'a pas pu être calculée. Pour filtrer sur un résultat agrégé calculé sur un groupe, il faut obligatoirement utiliser la clause HAVING qui intervient APRÈS le GROUP BY.

Question étudiant : Que se passe-t-il si j'oublie d'inclure une colonne présente dans le SELECT à l'intérieur de la clause GROUP BY ?
Réponse du professeur : Le SGBD renverra une erreur de syntaxe ou un comportement indéterminé. Si vous demandez d'afficher le nom de l'étudiant et la moyenne de ses notes en regroupant uniquement par numéro de promotion (GROUP BY id_promo), le SGBD ne sait pas quel nom afficher parmi tous les étudiants de la promotion pour une ligne de résultat donnée. La règle est absolue : toute colonne projetée dans le SELECT doit être soit regroupée (présente dans le GROUP BY), soit agrégée (dans un AVG, COUNT, SUM, etc.).

■ Exercice & Démonstration d'Application

Énoncé : Soit la base de données relationnelle de l'IUT composée des tables Etudiant(id_etud, nom, id_promo) et Note(id_note, id_etud, matiere, note). Écrire la requête SQL qui affiche le nom de chaque étudiant et sa moyenne dans la matière 'R1.05', uniquement pour les étudiants ayant au moins 2 notes enregistrées dans cette matière et une moyenne stricte supérieure à 12/20. Le résultat final doit être trié par moyenne décroissante.

Résolution pas à pas :
1. Identifier les tables sources et effectuer l'équijointure dans le FROM : FROM Etudiant E INNER JOIN Note N ON E.id_etud = N.id_etud. 2. Filtrer les n-uplets individuels sur la matière demandée avec le WHERE : WHERE N.matiere = 'R1.05'. 3. Structurer le regroupement par identifiant et nom d'étudiant : GROUP BY E.id_etud, E.nom. 4. Appliquer les conditions restrictives d'agrégation dans le HAVING : HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12. 5. Définir la projection dans la clause SELECT : SELECT E.nom, AVG(N.note) AS moyenne. 6. Définir le tri final du résultat avec la clause ORDER BY : ORDER BY moyenne DESC.

Résultat : SELECT E.nom, AVG(N.note) AS moyenne FROM Etudiant E INNER JOIN Note N ON E.id_etud = N.id_etud WHERE N.matiere = 'R1.05' GROUP BY E.id_etud, E.nom HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12 ORDER BY moyenne DESC;

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'application entre la clause WHERE et la clause HAVING en SQL ?

R : La clause WHERE filtre les lignes individuelles de la table avant le regroupement (sur des valeurs scalaires), tandis que la clause HAVING filtre les groupes déjà formés par la clause GROUP BY (sur des valeurs agrégées).

Q : Quel est le résultat produit par une requête avec LEFT JOIN si un enregistrement de la table de gauche ne possède aucune correspondance dans la table de droite ?

R : L'enregistrement de la table de gauche est conservé dans le résultat final, et l'ensemble des attributs provenant de la table de droite reçoivent la valeur NULL.

Q : Si une table A contient 10 enregistrements et une table B contient 5 enregistrements, combien de lignes retourne un produit cartésien (CROSS JOIN) entre ces deux tables ?

R : Un produit cartésien combine chaque ligne de A avec toutes les lignes de B, retournant exactement 10 x 5 = 50 lignes.